Deployment

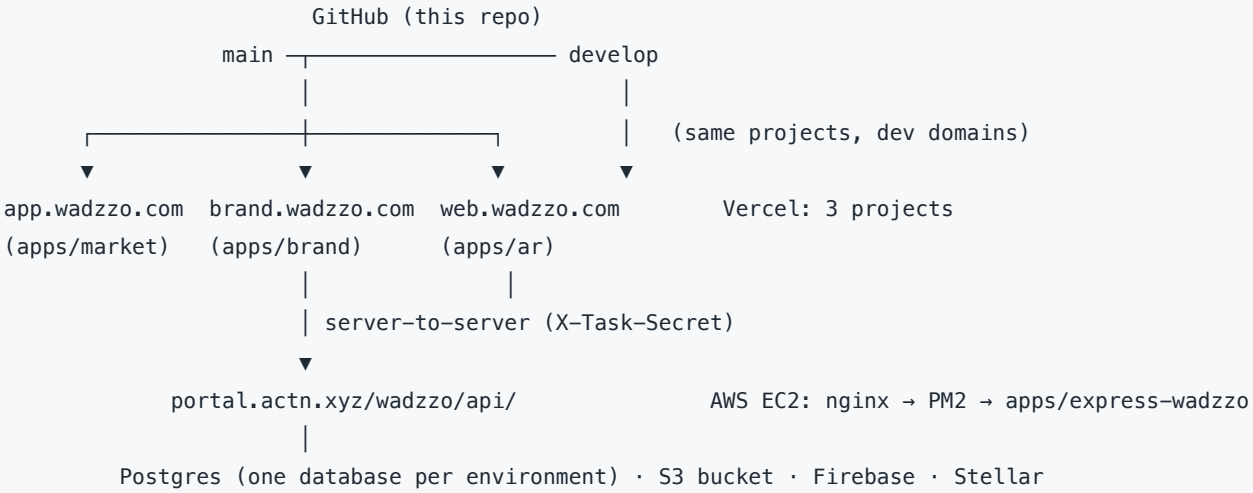
Generated 2026-10-13 from docs/DEPLOYMENT.md · 325 lines of source

Contents

1. Environments
2. Vercel: one project per app
3. Database
4. Task server on AWS
5. First deployment, in order
6. Releases
7. White-label platforms (Clinton County)
8. Troubleshooting

What this guide does not cover

The three web apps run on **Vercel**, the task server runs on **AWS (EC2)**. Everything is deployed from this one repository.



Piece	Where	Deployed by
Fan marketplace	Vercel project `wadzzo-market`	Vercel Git integration
Brand portal	Vercel project `wadzzo-brand`	Vercel Git integration
AR collector + mobile API	Vercel project `wadzzo-ar`	Vercel Git integration
Task server	AWS EC2, PM2, behind nginx	`.github/workflows/deploy-express.yml`
Mobile app	App Store / Play (Expo)	EAS, outside this guide

The names `wadzzo-market`, `wadzzo-brand`, `wadzzo-ar` are suggestions; use whatever your Vercel account already has.

1. Environments

Environment	Git branch	Web apps (domains)	Network	Database	Task server
Production	`main`	`app.wadzzo.com`, `brand.wadzzo.com`, `web.wadzzo.com`	mainnet	production Postgres	AWS EC2 (this guide)

Environment	Git branch	Web apps (domains)	Network	Database	Task server
Development	`develop`	`dev-app.wadzzo.com`, `dev-brand.wadzzo.com`, `dev-web.wadzzo.com`	testnet	development Postgres	a development task server, or none
Preview	any other	`*.vercel.app` (one per push)	testnet	development Postgres	none: set `EXPRESS_SERVER_URL` to a dead address
Local	-	`localhost:3000/3001/3002`	testnet	`pnpm db:local` or a dev DB	`pnpm --filter @wadzzo/express dev`

The `dev-*` names are a proposal (they need no wildcard DNS and sit under the apex you already own); pick another scheme if you prefer, the steps are the same.

Rules that keep the environments apart:

- A deployment is **one network**: `NEXT_PUBLIC_STELLAR_NETWORK` is `mainnet` in Production and `testnet` everywhere else. There is no runtime switch.
- The three apps of one environment share **one database, one bucket, the same platform wallets and the same issuer key**. Different environments share nothing.
- `ISSUER_ENCRYPTION_KEY` of Development must be a different key from Production's. It must be identical in the three apps of its own environment.
- On Vercel every deployment, previews included, runs with `NODE_ENV=production`. The code therefore uses `VERCEL_ENV` to decide what "production" means (for example, the task server's built-in production URL is used only when `VERCEL_ENV=production`).

2. Vercel: one project per app

2.1 Create the projects (three times)

Vercel → **Add New…** → **Project** → import this repository, then:

Setting	`wadzzo-market`	`wadzzo-brand`	`wadzzo-ar`
Framework preset	Next.js	Next.js	Next.js
Root Directory	`apps/market`	`apps/brand`	`apps/ar`
Include source files outside of the Root Directory	on	on	on
Node.js Version	20.x	20.x	20.x
Production Branch	`main`	`main`	`main`

The remaining behaviour is committed in each app's `vercel.json`: build command (`pnpm run build`, which runs `sync:platform` first), the ignored-build step (a push that doesn't touch the app or a package it depends on does not rebuild it), `framework`, and a 60 s function limit.

Add this environment variable to **all three** projects (Production, Preview, Development), so Vercel uses the pnpm version pinned in `package.json`:

```
ENABLE_EXPERIMENTAL_COREPACK=1
```

Check the first build log for `pnpm 12.x`.

2.2 Domains

Per project → **Settings** → **Domains**:

Project	Production domain	`develop` branch domain
`wadzzo-market`	`app.wadzzo.com`	`dev-app.wadzzo.com`
`wadzzo-brand`	`brand.wadzzo.com`	`dev-brand.wadzzo.com`
`wadzzo-ar`	`web.wadzzo.com`	`dev-web.wadzzo.com`

Add each domain, then for the dev ones choose **Git Branch** → `develop`. At your DNS provider create the record Vercel shows for each (a CNAME for subdomains). `www.wadzzo.com` should redirect to `app.wadzzo.com` (add it in the market project with "Redirect to").

Create the `develop` branch once: `git switch -c develop && git push -u origin develop`.

2.3 Environment variables

Each app's `apps/<app>/.env.example` lists every variable it reads, grouped, with what it is for. Enter them in **Settings** → **Environment Variables**:

- Values for Production → environment **Production**.
- Development values → environment **Preview**, scoped to the `develop` branch.
- Anything left on plain "Preview" (other branches) should hold only values that are safe for a throwaway deployment, and must not hold `NEXTAUTH_URL` (NextAuth then falls back to the preview's own URL).

Quickest way: `vercel env add`, or paste a whole `.env` block into the dashboard's "Import .env" (it accepts the sectioned files in `apps/<app>/.env.production`).

Values that differ between Production and Development:

Variable	Production (`main`)	Development (`develop`)
`PLATFORM_SLUG`	`wadzzo`	`wadzzo`
`NEXT_PUBLIC_URL` , `NEXTAUTH_URL`	the app's production domain	the app's `dev-*` domain
`NEXT_PUBLIC_STELLAR_NETWORK`	`mainnet`	`testnet`
`NEXT_PUBLIC_STAGE`	`prod` (real network and platform fees)	`dev`
`NEXT_PUBLIC_ASSET_CODE` / `_ISSUER`	the mainnet platform token	a testnet token
`NEXT_PUBLIC_HOME_DOMAIN`	`app.wadzzo.com`	`dev-app.wadzzo.com`
`MOTHER_SECRET` , `STORAGE_SECRET`	mainnet accounts	funded **testnet** accounts
`ISSUER_ENCRYPTION_KEY`	production key	**a different key**
`DATABASE_URL`	production Postgres	development Postgres
`DATABASE_POOL_MAX`	`1` (see section 3)	`1`
`AWS_BUCKET_NAME` and keys	production bucket	a separate bucket
`NEXTAUTH_SECRET`	production secret	a different secret
`SQUARE_*` , `NEXT_PUBLIC_SQUARE_*`	production credentials	**sandbox** credentials
`EXPRESS_SERVER_URL` (brand)	the production task server	a development task server, **always set**
`NEXT_PUBLIC_WADZZO_AR_URL` (brand)	`https://web.wadzzo.com`	`https://dev-web.wadzzo.com`

Variable	Production (`main`)	Development (`develop`)
`NEXT_PUBLIC_BRAND_PORTAL_URL` (market)	`https://brand.wadzzo.com`	`https://dev-brand.wadzzo.com`
`BRAND_APP_ORIGINS` (ar)	`https://brand.wadzzo.com`	`https://dev-brand.wadzzo.com`

Variables that must be **identical across the three apps of one environment**: `DATABASE_URL`, `ISSUER_ENCRYPTION_KEY`, `MOTHER_SECRET`, `STORAGE_SECRET`, `RUBY_ACCOUNTS_SECRET`, the asset code/issuer, the network, the bucket and its keys, the Firebase project and admin key, `PLATFORM_SLUG`. The task server's `NEXTAUTH_SECRET` must equal the **brand** app's.

Before you change anything in a dashboard, run the same check against your files: `pnpm env:check`.

2.4 Third-party allow-lists (the usual reasons a deploy "works but can't log in")

Add the six domains (three production, three `dev-*`) to:

Service	Where	Why
Firebase Authentication	Console → Authentication → Settings → **Authorized domains**	Google / Apple / Facebook popup sign-in
Sign in with Apple	Apple developer → Services ID → domains and return URLs	Apple sign-in on the web
Facebook login	App settings → valid OAuth redirect URIs	Facebook sign-in
WalletConnect	Cloud project → allowed domains	WalletConnect pairing
Square	Application → Web Payments allowed domains (market only)	card payments
Mapbox / Google Maps	token / key URL restrictions	maps
Stellar `home_domain`	the issuer account must point at `NEXT_PUBLIC_HOME_DOMAIN`	`stellar.toml` at `https://<home domain>/.well-known/stellar.toml` is served by the market app

2.5 What a Vercel deploy does

1. Install at the repository root (`pnpm install`, which also runs `prisma generate` through `@wadzzo/db`'s `postinstall`).
2. `pnpm run build` in the app: `sync:platform` (theme and assets for `PLATFORM_SLUG`), then `next build`.
3. Functions are created per route. The AR project ships the `onnxruntime-node` Linux binaries it needs (`outputFileTracingIncludes` in `apps/ar/next.config.js`).

Pushes to `main` promote to Production; pushes to `develop` update the `dev-*` domains. **Branch protection** on `main` and `develop` should require the `CI` check (`ci.yml`) so a red build can't deploy.

3. Database

- **One database per environment**, shared by the three apps and the task server.
- **Production runs on the EC2 Postgres** (`DATABASE_URL` host in your production files). Vercel functions have no fixed IP addresses, so the database's security group must accept port 5432 from Vercel (open it with TLS enforced and a strong password, or use Vercel's Static IPs if your plan includes them).

- **Connection count.** Serverless functions each hold their own connection. Set `DATABASE_POOL_MAX=1` on the Vercel projects and put **PgBouncer** (transaction mode) in front of Postgres, or Postgres's `max_connections` will be reached under load. The task server connects directly and may keep the default pool.
- **Schema changes** live only in `packages/db/prisma/schema.prisma`. Apply them from a trusted machine, never from a build:

  ```
  CONFIRM_DB_PUSH=yes DATABASE_URL="<env's url>" pnpm --filter @wadzzo/db push
  ```

  Order for a release that changes the schema: apply the (additive) schema change to the database first, then let the apps deploy. For a breaking change, deploy the code that no longer needs the old column first, then drop it.
- Development uses its own Postgres (Neon works well: its pooler endpoint is already pooled).

4. Task server on AWS

The task server (`apps/express-wadzzo`) is **one long-running Node process**. It keeps job state, server-sent-event streams and hotspot schedules in memory, so it must run as **exactly one instance** on an always-on machine; it cannot run on Vercel or any serverless platform, and a restart drops in-flight jobs (deploy when `GET /health` reports no `processing` jobs).

4.1 Machine

Item	Choice
Instance	EC2 `t3.small` or larger (2 GB RAM: the agent and `sharp` need headroom; PM2 restarts it above 512 MB), Ubuntu 24.04 LTS, 20 GB gp3
Address	An Elastic IP; DNS `A` record `portal.actn.xyz` → that IP (or any hostname you prefer)
Region	The same region as the production database
Security group	inbound `80` and `443` from anywhere; `22` from your own IP (and see 4.5 for CI); **do not open** `4000`

4.2 One-time setup

```
# as a sudo user on the instance
sudo adduser --disabled-password wadzzo && sudo usermod -aG sudo wadzzo   # or reuse an existing user
sudo apt update && sudo apt install -y nginx certbot python3-certbot-nginx git rsync curl

# Node 20 (matches .nvmrc), pnpm (from packageManager), PM2
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
sudo corepack enable
sudo npm install -g pm2

# app directory (EC2_APP_PATH in the workflow)
sudo mkdir -p /opt/wadzzo && sudo chown wadzzo:wadzzo /opt/wadzzo
```

Then, as `wadzzo` :

```
mkdir -p /opt/wadzzo/apps/express-wadzzo
$EDITOR /opt/wadzzo/apps/express-wadzzo/.env    # paste apps/express-wadzzo/.env.example, fill in the real values
chmod 600 /opt/wadzzo/apps/express-wadzzo/.env
```

This file lives only on the server: the deploy workflow never writes or overwrites `.env*`.

Required values (the file explains each): `NEXTAUTH_SECRET` (**the production brand app's value**), `DATABASE_URL` (the production database), `OPENAI_API_KEY`, `GOOGLE_MAP_API_KEY`, and `AWS_BUCKET_NAME`, `AWS_BUCKET_REGION`, `AWS_ACCESS_KEY`, `AWS_SECRET_ACCESS_KEY`. Set `NODE_ENV=production`. Leave `ALLOWED_ORIGINS` empty: only the brand app's server calls this service.

4.3 nginx and TLS

```
sudo cp apps/express-wadzzo/deploy/nginx-wadzzo-api.conf /etc/nginx/sites-available/wadzzo-api
sudo ln -s /etc/nginx/sites-available/wadzzo-api /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d portal.actn.xyz        # adds HTTPS and auto-renewal
```

The config serves the task server at `https://portal.actn.xyz/wadzzo/api/`, strips the prefix, turns buffering off for the SSE stream and allows 50 MB bodies. That URL is also the production default inside the brand app, so you don't have to set `EXPRESS_SERVER_URL` in Production, but **do set it anyway** so it is visible and explicit.

4.4 First start

Do this after the secrets in 4.5 exist.

1. GitHub → **Actions** → **Deploy task server (AWS EC2)** → **Run workflow** on `main`. It syncs the workspace to the server, installs, builds, starts the process under PM2 and waits for `/health`.
2. On the server, make PM2 survive a reboot and rotate its logs:

   ```
   pm2 startup systemd          # run the command it prints
   pm2 save
   pm2 install pm2-logrotate
   curl -fsS http://127.0.0.1:4000/health
   curl -fsS https://portal.actn.xyz/wadzzo/api/health
   ```

Every later deploy is a push to `main`.

4.5 Automatic deploys

`.github/workflows/deploy-express.yml` runs on a push to `main` that touches the task server or one of the packages it bundles (`agent-contract`, `errors`, `geo`, `storage`, the schema). It tests, builds, syncs the workspace to the server (never `.env*`, `node_modules` or `.git`), installs, restarts under PM2 and waits for `/health`.

GitHub → **Settings** → **Secrets and variables** → **Actions** (a `production` environment is recommended, so you can require an approval):

Secret	Value
`EC2_HOST`	the Elastic IP or hostname
`EC2_USER`	`wadzzo`
`EC2_SSH_KEY`	a private key whose public half is in `~/.ssh/authorized_keys`

Secret	Value
`EC2_APP_PATH`	`/opt/wadzzo`

GitHub's runners connect from changing addresses, so port 22 has to accept them. Either keep key-only SSH open on 22 (disable password logins, install `fail2ban`), or restrict 22 and deploy through AWS Systems Manager instead.

4.6 Operating it

Task	Command
Logs	`pm2 logs express-server` (the service logs JSON lines to stdout; PM2 keeps them, `pm2-logrotate` trims them)
Status, memory	`pm2 status`, `curl -s http://127.0.0.1:4000/health`
Restart	`pm2 restart express-server`
Roll back	revert the commit on `main`; the workflow redeploys the previous build
Rotate the shared key	change `NEXTAUTH_SECRET` in the brand project **and** in `.env` here, redeploy both

Uptime monitoring: point any checker at `https://portal.actn.xyz/wadzzo/api/health`. It is unauthenticated and returns uptime, memory and job counts; restrict it in nginx if you don't want that public.

4.7 A development task server (optional)

The Development environment needs `EXPRESS_SERVER_URL` set explicitly. Options, cheapest first:

1. **None.** Set it to `http://127.0.0.1:9`. The brand features that need the task server (the map agent, hotspot drops) then fail clearly instead of touching production.
2. **A second PM2 app** on the same instance with `PORT=4001`, a different `.env` (development database, the **development** brand's `NEXTAUTH_SECRET`) and an extra nginx `location /wadzzo-dev/api/`. Point the development brand project at `https://portal.actn.xyz/wadzzo-dev/api`.

5. First deployment, in order

1. Database ready and schema applied (section 3). Buckets and Firebase projects exist.
2. Task server on AWS (section 4) and `/health` returns `ok`.
3. Create the three Vercel projects (2.1), add environment variables (2.3), domains (2.2), allow-lists (2.4).
4. Push `develop`; open the three `dev-*` domains and run the checks below.
5. Merge `develop` into `main`; open the three production domains and run the checks again.

Checks after any deployment

Check	Expect
`https://app.wadzzo.com/.well-known/stellar.toml`	the platform's organisation and currencies
`https://app.wadzzo.com` and `https://brand.wadzzo.com`	pages render with the right branding
`https://web.wadzzo.com/api/mobile/v1/bounties`	`200` JSON
Sign in with email, then with a wallet	a session, no redirect loop
Brand portal → create a drop with an image	the upload succeeds (bucket + keys)
Brand portal → map agent	a job starts and streams (task server + shared secret)

Check	Expect
An unauthenticated tRPC call that passes `signWith`	`401 AUTH_REQUIRED`

6. Releases

```
feature branch —PR→ develop —(verify on dev-* domains)→ main → production
```

- **Web apps:** Vercel keeps every deployment. **Rollback** is "Promote to production" on the previous deployment (instant, no rebuild).
- **Task server:** revert on `main` and let the workflow run, or run the workflow manually (Run workflow) with an older commit selected.
- **A change touching both** (for example, a new task-server route the brand app calls): deploy the task server first, then the app.

7. White-label platforms (Clinton County)

A partner is **the same code with another** `PLATFORM_SLUG`, not another branch. For each partner app create one more Vercel project from the same repository and Root Directory:

- `PLATFORM_SLUG=clintoncounty` (this picks the theme, assets and branding at build time).
- The partner's own domain, token, wallets and `TOML_ORG_*`.
- Only what differs is in `apps/<app>/.env.clintoncounty.production`; everything else is the Wadzzo project's value.
- The partner shares the Wadzzo database and task server (rows are scoped by platform).

See PLATFORMS.md for adding a platform.

8. Troubleshooting

Symptom	Cause and fix
Build fails with `CONFIG_MISSING` / `Invalid environment variables`	A required variable is missing for that environment (Production vs Preview). Compare with `.env.example`.
Login popup closes, or "unauthorized domain"	The domain isn't in Firebase's authorized domains (2.4).
Login works but the session is lost on the next request	`NEXTAUTH_URL` differs from the domain being used, or `NEXTAUTH_SECRET` differs between deployments.
Brand features that use the agent fail with "task server isn't configured"	`EXPRESS_SERVER_URL` is unset in a non-production environment (by design, see section 1).
Task server answers `401`	Brand's and the task server's `NEXTAUTH_SECRET` differ.
Uploads fail with `CONFIG_MISSING` naming `AWS_*`	Bucket variables missing. They are read as `AWS_*` or the older `NEXT_AWS_*`.
`too many connections` in Postgres logs	`DATABASE_POOL_MAX` not `1` on Vercel, or no PgBouncer (section 3).
A function times out at 60 s	Raise `maxDuration` in the app's `vercel.json` (plan limits apply), or move the work to the task server.
The AR project's function is too large	Check the build output for the `onnxruntime-node` / `@huggingface/transformers` bundle sizes; the AR app was sized for these.

Symptom	Cause and fix
`pnpm env:check` reports a mismatch	Fix the value in the file *and* in the Vercel project; the check reads files, Vercel holds the truth.

What this guide does not cover

- EAS builds and store releases for `apps/mobile`.
- Backups and point-in-time recovery of Postgres.
- Cost sizing of Vercel plans (function duration and bandwidth limits depend on your plan; check them against the `maxDuration` values in `vercel.json`).